goxpyriment — Installation

Contents

Goxpyriment installation instructions

There are three ways to use goxpyriment, depending on what you already have and what you want to do. Find your situation below and jump to the matching section — each choice builds on the previous one, so you only need to read as far as your goal requires.

Your situation	What you need	Go to
You were given a **ready-to-run app** (a compiled file) and just want to launch it	Nothing	No installation
You have the **source code** of an experiment (`.go` files) and want to run or build it	Go	Minimal installation
You want to **write your own experiments** or explore the framework	Go + Git + a code editor	Full installation

No installation is needed to run a goxpyriment app

If you were given a ready-to-run goxpyriment app (a compiled file, such as the ones provided in the compiled examples), you do not need to install anything to use it. Just launch the app by double-clicking its icon in your file folder, or by typing its name in your command line (Terminal or Command Prompt).

 WARNING One potential issue can arise from the antivirus or protection system of your computer if these binaries are unsigned: macOS Gatekeeper and Windows Defender will show security warnings or worse, *misleading messages* such as ‘this

program is damaged'. Your antivirus may quarantine the files. Don't let them intimidate you:

- macOS: Right-click the app → **Open**, or run `xattr -dr com.apple.quarantine <AppName>.app` in Terminal. See macOS installation and security for step-by-step instructions.
- Windows: Just click on "More info" then "Run anyway". (Note: These warnings will only pop out the first time you try to execute a given program).

Minimal installation to execute a goxpyriment source code

Let us consider the case where you have the *source code* of a goxperiment program.

If Go is not already installed on your computer (you can check if Go installed by typing `go version` on a command line. If an error message is displayed, then Go is not installed), download and install it following the instructions at https://go.dev/doc/install

Then there are two possibilities. You have either:

1. A folder containing at least the files `go.mod`, `go.sum` and the source code, say `experiment.go` (it can be any file with the `.go` extension).

   Then, execute the command line:

   ```
   go run experiment.go    # replace experiment by the actual name
   ```

   Or build the app, then run it, by typing:

   ```
   go build experiment.go
   ./experiment
   ```

2. A single `.go` file, say `experiment.go`.

   Then, you need to copy this file into a folder, e.g. `experiment` and create `go.mod` and `go.sum` by typing

   ```
   go mod init experiment
   go mod tidy
   ```

   This will create the missing files and you are in the situation of the previous paragraph: you can run the program with the command `go run .` or `go build .`.

Remark: the very first time you run or build a goxpyriment program, it will take a while as Go needs to download a number of modules from the Internet. Afterwards, running or building an app should be near instantaneous.

Full installation

If you want to do serious development, in addition to Go, you need to install Git, a code editor and, probably, an AI-coding agent like Claude Code, Gemini cli or Aider. If you are new to this, consult the detailed instructions

Then:

1. Clone goxpyriment Github repository, by opening a shell (e.g. launch `Git Bash` under Windows), and executing the command-line

   ```
   git clone https://github.com/chrplr/goxpyriment.git
   ```

   This will create a folder `goxpyriment` contain the entire source code, examples and documentation of the framework. In the future, you will be able run the `git pull` command within this folder to update to the latest version.

2. Navigate to the `goxpyriment` folder and type the command-line:

   ```
   ./build-all.sh
   ```

   (Note: On Linux/macOS you can instead use `make all`)

   If all goes well, this will compile codes from examples/* and tests/* (Note: The first time, this will take a while because Go needs to download several modules. Once this is done, future compilations will be fast.)

   After this operation, a new `_build` folder contains executable apps for many experiments. You can either run them from the command line, or launch them by clicking on their icon in the folder.

3. If you would like to read the documentation locally — both the API reference (via pkgsite) and this site (via zensical) — see Viewing the documentation locally.

One extra step on a Linux machine used for data collection

Nothing above requires special privileges, and you can write and run experiments without doing any of this. But an experiment that has to present stimuli on time competes with everything else on the machine, and the operating system has no reason to prefer it — so on a machine that will actually collect data, grant your user real-time scheduling before you start trusting its timing.

The procedure is in Setting priority under Linux: create a group, add a file to `/etc/security/limits.d/`, add yourself to the group, then **log out and log back in**.

Check the file afterwards with `grep rtprio /etc/security/limits.d/*.conf`, and check the grant is live with `ulimit -r` after logging back in — it should print your chosen priority, not `0`.

Once that is in place you can verify the machine's timing end to end with Timing Tests, which also lists the other Linux tuning worth doing (disabling the compositor is the single largest improvement).

Program your own experiments

Follow the step-by-step guide in Creating Your Own Experiment, which walks you through writing, running, building, and sharing your own experiment.

For background and reference, see Getting Started, the examples' source codes, and the available functions.

Use an AI coding agent

If you launch `claude` (Claude Code) from the `goxpyriment` folder, the many `CLAUDE.md` files will help the AI coding agent to create experiments for you. You can describe the experimental paradigm in plain language (see the `description.md` files in `examples` subfolders) and ask claude code to program it using the goxpyriment framework.